

In Class Assignment One

In the following assignment you will be asked to fill in python code and derivations for a number of different problems. Please read all instructions carefully and turn in the rendered notebook (or HTML of the rendered notebook) before the end of class (or right after class). The initial portion of this notebook is given before class and the remainder is given during class. Please answer the initial questions before class, to the best of your ability. Once class has started you may rework your answers as a team for the initial part of the assignment.

Contents

- Loading the Data
- Linear Regression
- For in class session (not included here):
- Using Scikit Learn for Regression
- Linear Classification

Back to Top

Loading the Data

Please run the following code to read in the “diabetes” dataset from sklearn’s data loading module.

This will load the data into the variable `ds`. `ds` is a **bunch** object with fields like `ds.data` and `ds.target`. The field `ds.data` is a numpy matrix of the continuous features in the dataset. **The object is not a pandas dataframe. It is a numpy matrix.** Each row is a set of observed instances, each column is a different feature. It also has a field called `ds.target` that is a continuous value we are trying to predict. Each entry in `ds.target` is a label for each row of the `ds.data` matrix.

The diabetes dataset was first used in the following paper: - <http://statweb.stanford.edu/~imj/WEBLIST/2004/LarsAnnStat04.pdf>

```
library(reticulate)
py_require("numpy")
py_require("scikit-learn")

from sklearn.datasets import load_diabetes
import numpy as np

ds = load_diabetes()

# this holds the continuous feature data
# because ds.data is a matrix, there are some special properties we can access (like 'shape')
# there are 442 instances and 10 features per instance
print('features shape:', ds.data.shape, 'format is:', ('rows','columns'))

## features shape: (442, 10) format is: ('rows', 'columns')
print('range of target:', np.min(ds.target),np.max(ds.target))

## range of target: 25.0 346.0

from pprint import pprint

# we can set the fields inside of ds and set them to new variables in python
pprint(ds.data) # prints out elements of the matrix
```

```
## array([[ 0.03807591,  0.05068012,  0.06169621, ..., -0.00259226,
##          0.01990749, -0.01764613],
##        [-0.00188202, -0.04464164, -0.05147406, ..., -0.03949338,
##          -0.06833155, -0.09220405],
##        [ 0.08529891,  0.05068012,  0.04445121, ..., -0.00259226,
##          0.00286131, -0.02593034],
##        ...,
##        [ 0.04170844,  0.05068012, -0.01590626, ..., -0.01107952,
##          -0.04688253,  0.01549073],
##        [-0.04547248, -0.04464164,  0.03906215, ...,  0.02655962,
##          0.04452873, -0.02593034],
##        [-0.04547248, -0.04464164, -0.0730303 , ..., -0.03949338,
##          -0.00422151,  0.00306441]], shape=(442, 10))
```

```
pprint(ds.target) # prints the vector (all 442 items)
```

```
## array([151.,  75., 141., 206., 135.,  97., 138.,  63., 110., 310., 101.,
##        69., 179., 185., 118., 171., 166., 144.,  97., 168.,  68.,  49.,
##        68., 245., 184., 202., 137.,  85., 131., 283., 129.,  59., 341.,
##        87.,  65., 102., 265., 276., 252.,  90., 100.,  55.,  61.,  92.,
##        259.,  53., 190., 142.,  75., 142., 155., 225.,  59., 104., 182.,
##        128.,  52.,  37., 170., 170.,  61., 144.,  52., 128.,  71., 163.,
##        150.,  97., 160., 178.,  48., 270., 202., 111.,  85.,  42., 170.,
##        200., 252., 113., 143.,  51.,  52., 210.,  65., 141.,  55., 134.,
##        42., 111.,  98., 164.,  48.,  96.,  90., 162., 150., 279.,  92.,
##        83., 128., 102., 302., 198.,  95.,  53., 134., 144., 232.,  81.,
##        104.,  59., 246., 297., 258., 229., 275., 281., 179., 200., 200.,
##        173., 180.,  84., 121., 161.,  99., 109., 115., 268., 274., 158.,
##        107.,  83., 103., 272.,  85., 280., 336., 281., 118., 317., 235.,
##        60., 174., 259., 178., 128.,  96., 126., 288.,  88., 292.,  71.,
##        197., 186.,  25.,  84.,  96., 195.,  53., 217., 172., 131., 214.,
##        59.,  70., 220., 268., 152.,  47.,  74., 295., 101., 151., 127.,
##        237., 225.,  81., 151., 107.,  64., 138., 185., 265., 101., 137.,
##        143., 141.,  79., 292., 178.,  91., 116.,  86., 122.,  72., 129.,
##        142.,  90., 158.,  39., 196., 222., 277.,  99., 196., 202., 155.,
##        77., 191.,  70.,  73.,  49.,  65., 263., 248., 296., 214., 185.,
##        78.,  93., 252., 150.,  77., 208.,  77., 108., 160.,  53., 220.,
##        154., 259.,  90., 246., 124.,  67.,  72., 257., 262., 275., 177.,
##        71.,  47., 187., 125.,  78.,  51., 258., 215., 303., 243.,  91.,
##        150., 310., 153., 346.,  63.,  89.,  50.,  39., 103., 308., 116.,
##        145.,  74.,  45., 115., 264.,  87., 202., 127., 182., 241.,  66.,
##        94., 283.,  64., 102., 200., 265.,  94., 230., 181., 156., 233.,
##        60., 219.,  80.,  68., 332., 248.,  84., 200.,  55.,  85.,  89.,
##        31., 129.,  83., 275.,  65., 198., 236., 253., 124.,  44., 172.,
##        114., 142., 109., 180., 144., 163., 147.,  97., 220., 190., 109.,
##        191., 122., 230., 242., 248., 249., 192., 131., 237.,  78., 135.,
##        244., 199., 270., 164.,  72.,  96., 306.,  91., 214.,  95., 216.,
##        263., 178., 113., 200., 139., 139.,  88., 148.,  88., 243.,  71.,
##        77., 109., 272.,  60.,  54., 221.,  90., 311., 281., 182., 321.,
##        58., 262., 206., 233., 242., 123., 167.,  63., 197.,  71., 168.,
##        140., 217., 121., 235., 245.,  40.,  52., 104., 132.,  88.,  69.,
##        219.,  72., 201., 110.,  51., 277.,  63., 118.,  69., 273., 258.,
##        43., 198., 242., 232., 175.,  93., 168., 275., 293., 281.,  72.,
##        140., 189., 181., 209., 136., 261., 113., 131., 174., 257.,  55.,
##        84.,  42., 146., 212., 233.,  91., 111., 152., 120.,  67., 310.,
```

```
##          94., 183., 66., 173., 72., 49., 64., 48., 178., 104., 132.,
##          220., 57.]])
```

[Back to Top](#)

Using Linear Regression

In the videos, we derived the formula for calculating the optimal values of the regression weights (you must be connected to the internet for this equation to show up properly):

$$w = (X^T X)^{-1} X^T y$$

where X is the matrix of values with a bias column of ones appended onto it. For the diabetes dataset one could construct this X matrix by stacking a column of ones onto the `ds.data` matrix.

$$X = \begin{bmatrix} & \vdots & & 1 \\ \dots & \text{ds.data} & \dots & \vdots \\ & \vdots & & 1 \end{bmatrix}$$

Question 1: For the diabetes dataset, how many elements will the vector w contain?

Linear Regression Notes

From video 0 min - 2 min * $y = \text{ds.target}$ (ranges from 1 to m) * $m = 442$ (length of `ds.target`) * $x = \text{ds.data}$ (m vectors of length N) * N (number of instances) = 10 (length of vectors in `ds.data`) * $w =$ unknown linear weights (length equal to $N =$ number of elements in each X) * Then $\hat{y}^{(i)} = w^\top x^{(i)}$

From video 14 min - 20 min

To be precise: * Suppose $\vec{y}$ is the column vector with $m = 442$ elements * Suppose X is the matrix with each row vector $x^{(i)}$ is length $N = 10$, and it has $m = 442$ rows * Then $w = (X^T X)^{-1} X^T y$

From video 21 min - 31 min:

- Unfortunately, $(X^T X)^{-1}$ may not exist.
- In that case we can use gradient descent iteratively from some guess value W_0 :

$$W - = \alpha \sum_i (y^{(i)} - w^\top x^{(i)}) x^{(i)}$$

- There are some techniques that add terms to this update function for different “flavors” of linear regression

Classification Notes

- In the case of binary classification, we say $\hat{y}^{(i)} = g(w^\top x^{(i)})$ such that the function g always returns 0 or 1. One such example is:

```
def g(theta): return 1 if theta >= 0 else 0
```

- This function is not strictly speaking differentiable. It can be represented as the Dirac delta function:

$$\delta(x) = \begin{cases} 0, & x \neq 0 \\ \infty, & x = 0 \end{cases}, \int_{-\infty}^{\infty} \delta(x) dx = 1$$

From video 37 min - 45 min:

- As a result, we'd like to find a g with a more well-behaved derivative. One such example is a logistic curve:

```
def g(theta): return 1 / (1 + np.exp(-theta))
```

- This time, the derivative is $g(\theta)(1 - g(\theta))$ which is well behaved.
- Our update function this time is:

$$W_j = \alpha \sum_i (y^{(i)} - w^\top x^{(i)}) (x_j^{(i)}) g(w^\top x^{(i)}) (1 - g(w^\top x^{(i)}))$$

- This is called “logistic regression” due to its similarity, but it is **not** a linear regression problem!

```
m = ds.data.shape[0]
N = ds.data.shape[1]
y = ds.target
# Add one for the bias term
print(N + 1)
```

```
## 11
```

Exercise 1: In the following empty cell, use the given equation above (using numpy matrix operations) to find the values of the optimal vector w . You will need to be sure X and y are created like the instructor talked about in the video. Don't forget to include any modifications to X to account for the bias term in w . You might be interested in the following functions:

- `import numpy as np`
- `np.hstack((mat1,mat2))` stack two matrices horizontally, to create a new matrix
- `np.ones((rows,cols))` create a matrix full of ones
- `my_mat.T` takes transpose of numpy matrix named `my_mat`
- `np.dot(mat1,mat2)` or `mat1 @ mat2` is matrix multiplication for two matrices
- `np.linalg.inv(mat)` gets the inverse of the variable `mat`

```
# Write you code here, print the values of the regression weights
# using the 'print()' function in python
import numpy as np
```

```
def matrix_trans(matrix): return matrix.T
def matrix_inv(matrix): return np.linalg.inv(matrix)
```

```
# Add bias term to matrix and test
bias = np.ones((m, 1))
X = np.hstack((ds.data, bias))
#pprint(X[0])
```

```
# Then $w = (X^TX)^{-1}X^Ty$
X_T = matrix_trans(X)
w = matrix_inv(X_T @ X) @ X_T @ y
print(w)
```

```
## [ -10.0098663  -239.81564367  519.84592005  324.3846455  -792.17563855
##    476.73902101  101.04326794  177.06323767  751.27369956   67.62669218
##    152.13348416]
```